

Specyfikacja Projektu Zaliczenowego (Programowanie w Języku Java) Mini Git - Glit

Weronika Szumilas, Jan Gryglaszewski

14.04.2026



Rysunek 1: Logo projektu wygenerowane przy użyciu narzędzia Microsoft Designer.

1. Tytuł Projektu

Glit - nazwa nawiązuje do jednego z najpopularniejszych systemów kontroli wersji jednocześnie nawiązując do słowa *to glitch* - zaciąć się, przestać działać (na chwilę), podkreślając skromność autorów świadomych zaawansowania oryginału, do którego jedynie mogą próbować dążyć przy realizacji swojej mocno okrojonej wersji.

2. Opis i Cele Projektu

Projekt ma na celu stworzenie narzędzia uproszczonego systemu kontroli wersji, który umożliwi użytkownikom tworzenie lokalnego repozytorium oraz pracy na kilku niezależnych gałęziach.

Główne cele projektu:

- Dostęp do poprzednich wersji plików.
- Praca na kilku różnych gałęziach.
- Merge kilku gałęzi.

3. Wymagania Funkcjonalne

- Inicjalizacja lokalnego repozytorium (init).
- Dodawanie/usuwanie plików do/z przestrzeni śledzonej (add/rm).
- Wyświetlanie stanu indexu ((un)tracked, changes (not) staged, itd.) (status).
- Dodawanie aktualnej wersji śledzonych plików do repozytorium (commit).
- Tworzenie nowych gałęzi (branch).
- Łączenie gałęzi (merge).
- Przeglądanie zmian między wersjami (log).

4. Wymagania Niefunkcjonalne

- **Wydajność:** Czas przetwarzania danych poniżej 10 sekund.
- **Skalowalność:** Obsługa minimum 5 gałęzi.
- **Zgodność:** można na dowolnym urządzeniu z zainstalowanym JDK.

5. Architektura Systemu

Typ architektury: **Monolit**.

Schemat:

- Backend: Java.
- Baza danych: lokalna struktura plików.

6. Zastosowane Algorytmy

- **Algorytm liczenia sumy kontrolnej:** Liczenie sumy kontrolnej treści pliku.
- **Algorytmy mergowania bez/z konfliktami:** Łączenie gałęzi.
- **Graf skierowany acykliczny:** Kolejne commity są potomkami poprzednich wersji (commitów).
- **Kompresja plików:** Treść plików przechowywana w skompresowanej wersji.

7. Struktura Klas

Model

- Blob - hash, dane pliku.
- Tree - drzewo historii projektu (struktura katalogów i plików).
- Commit - parent, author, timeStamp, message.

Serwis

- GlitService - logika zarządzania komendami.

Kontroler

- GlitController - obsługa komend linii poleceń (wejście/wyjście).

Kluczowe komponenty systemu

- ObjectDatabase - odpowiada za zapis i odczyt obiektów (Blob, Tree, Commit) w katalogu .glit/objects.
- Index - przechowuje aktualny stan plików przygotowanych do commita (staging area).
- RefManager - zarządza referencjami (HEAD, branche).
- TreeBuilder - buduje strukturę drzewa na podstawie plików z Index.
- MergeEngine - odpowiada za łączenie gałęzi oraz wykrywanie konfliktów.

8. Wykorzystane Technologie i Biblioteki

Backend:

Java (JDK 17)

Biblioteki standardowe:

- java.nio.file – operacje na systemie plików
- java.security – obliczanie hashy (SHA-1)
- java.util.zip – kompresja danych

Baza danych:

Brak – dane przechowywane w strukturze plików (.glit)

Narzędzia:

- Git – kontrola wersji projektu
- Maven – zarządzanie projektem
- JUnit – testy jednostkowe

9. API i Modele Danych

Aplikacja działa jako narzędzie CLI (Command Line Interface), dlatego nie posiada klasycznego REST API.

Komendy CLI:

```
glit init
glit add <plik>
glit rm <plik>
glit status
glit commit -m "message"
glit branch <name>
glit checkout <branch>
glit merge <branch>
glit log
```

10. Interfejs Użytkownika

Aplikacja posiada interfejs tekstowy (CLI).

Cechy:

- Obsługa komend w terminalu
- Czytelne komunikaty błędów
- Prosta składnia poleceń

11. Etapy Realizacji / Harmonogram

Faza	Opis	Czas
Planowanie	Określenie wymagań, wybór technologii	1 tydzień
Backend	Implementacja logiki systemu (Blob, Tree, Commit, ObjectDatabase, Index, operacje repozytorium)	4 tygodnie
Testy i realtime	Testy jednostkowe i integracyjne	1 tydzień

12. Testowanie

- Testy jednostkowe (logika backendu).
- Testy obciążeniowe – obsługa wielu plików na raz.

13. Ryzyka i Środki Zaradcze

Ryzyko	Wpływ	Zarządzanie
Błędna implementacja zapisu obiektów (ObjectDatabase)	Wysoki	Testy zapisu i odczytu, walidacja hashy, unikanie nadpisywania istniejących obiektów
Utrata danych w repozytorium	Wysoki	Przechowywanie danych w sposób niezmienny (immutable), testy integralności danych
Błędy w algorytmie merge	Wysoki	Uproszczony algorytm merge, wykrywanie konfliktów zamiast automatycznego rozwiązywania
Niepoprawne zarządzanie referencjami (HEAD, branche)	Średni	Testy operacji checkout i branch, walidacja aktualnego stanu repozytorium
Kolizje hashy (SHA-1)	Niski	Użycie sprawdzonego algorytmu SHA-1, dodatkowa walidacja zawartości
Złożoność projektu	Średni	Ograniczenie funkcjonalności do podstawowych operacji (init, add, commit, branch, merge)
Błędy w indeksie (staging area)	Średni	Spójność indexu z systemem plików, testy dodawania i usuwania plików

14. Kryteria Sukcesu

- Wszystkie funkcje działają zgodnie ze specyfikacją.
- Pokrycie testami > 70%.
- Brak błędów krytycznych na produkcji.
- Pozytywna opinia użytkowników (średnia ocena >4.5).

15. Szacunkowe Nakłady Pracy (Workload Estimation)

Całkowity szacowany czas pracy: **150 godzin roboczych**.

Moduł / Zadanie	Opis	Szac. czas (h)
-----------------	------	----------------

Analiza i projektowanie	Analiza wymagań, wybór technologii	15
Programowanie	Modele danych, logika działania, skrypt uruchomieniowy.	80
Testowanie	Pokrycie testami kluczowych ścieżek, testy obciążeniowe	20
Dokumentacja	Dokumentacja techniczna i instrukcja użytkownika	10
Bufor czasowy / błędy	Debugowanie, nieprzewidziane problemy	25